PROMISES

HANDBOOK

BEGINNER - INTERMEDIATE

@krishnadamaraju

Terms to know

Synchronous

Occurring at the present moment.

Asynchronous

Not occuring at the present moment.

Non-blocking

If your program or a piece of code doesnt block/pause the execution of the next program/code, it is non-blocking. This is a OS level concept.

Settled

When you get any sort of response (Success/Failure) from a PROMISE it is known to be in Settled stated. AKA Promise is done

Fulfilled

When you get a success response from a PROMISE, then it is said to be FULFILLED

What is a Promise

#ExplainLikeIamFive
Similar to human promises, Js Promises are something that would ASSURE you to get you something in some amount of time. So, they don't happen right now, hence they are **Asynchronous**.

Def: The Promise object represents the eventual completion (or failure) of an asynchronous operation, and its resulting value. (Source: MDN)

```javascript
function getPromise() {
  return new Promise((resolve, reject) => {
    resolve('Promise is Successful/Fulfilled');
  })
}

getPromise().then((resp) => {console.log(resp)})
//Promise is Successful/Fulfilled
```

- Every Promise accepts only one function called **Executor function, whose parameters are the Reject and Resolved callbacks**

- Promise is a **Constructor function** in Js. Hence it can return prototypes or helper methods.

FAQ

How do I know about the Status of the promise?

Promise has three states / Status - **Pending**, **Resolved** and **Rejected**.

Can I return the data from promise

Yes, only with the callback functions provided to the executor function. In the Example below, **resolve** and **reject** are the callbacks based on the state.

```javascript
function getPromise() {
  return new Promise((resolve, reject) => {
    resolve('Promise is Successful/Fulfilled');
  })
}

getPromise().then((resp) => {console.log(resp)})
//Promise is Successful/Fulfilled
```

FAQ

How are Promise States related to the callback functions of Executor function ?

Callback functions can be named anything, and the first callback function ideally accept the **Success/Fulfilled state** and Second for the **Rejected/Error** state. There may be other ways of using, but this way is mostly used.

What is the use of then() ?

Promises have two types of methods that we can use. **Instance methods and Static Methods.**

then() is one of the instance methods, that is **ideally** used to capture the **resolved** from the Promise.

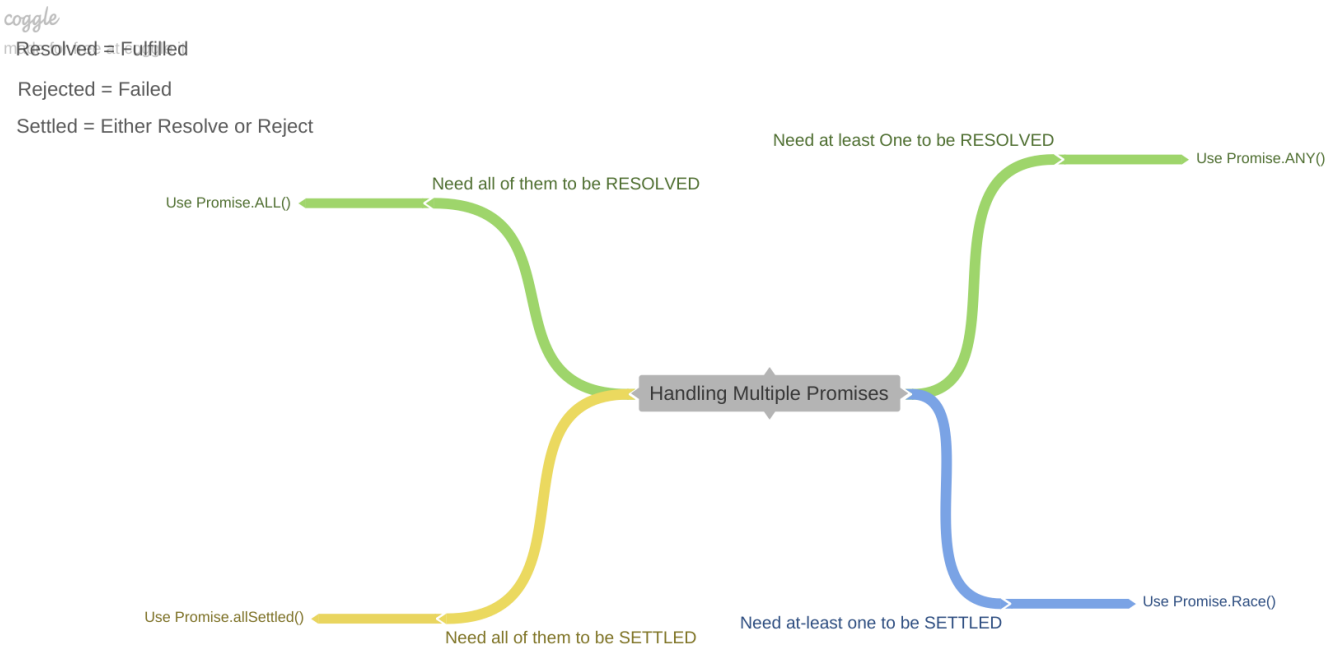
Similarly, **catch()** to capture Errors and **finally()** when the promise is resolved.

Usage of then is called as **Thenable**

FAQ

What are Static methods ?

Some **methods** of Promise can be **used without** invocation of a **Promise instance**, that is using `**new Promise(/* ... */)**`. Such methods are called **Static Methods** and these methods can be used to handle on multiple promise instances.



Usage	Static Method	Input	Returns
Need all Promises to be Successful	Promise.all()	Array	A Promise
Need all promises to be settled (either success or failure)	Promise.allSettled()	Array	A Promise
Need at least one of the promise to be successful	Promise.any()	Array	A Promise
Need at least one of the promise is settled (either success or failure)	Promise.race()	Array	A Promise
	Promise.reject()		A Promise
	Promise.resolve()		A Promise

FAQ

What are Instance methods ?

A promise instance is when a promise is created with **new** keyword. When such instance is created, there are some methods at are bound to the created instance and can be used to let the **status** of the Instance.

Instance methods

`Promise.prototype.catch()`

Appends a rejection handler callback to the promise, and returns a new promise resolving to the return value of the callback if it is called, or to its original fulfillment value if the promise is instead fulfilled.

`Promise.prototype.then()`

Appends fulfillment and rejection handlers to the promise, and returns a new promise resolving to the return value of the called handler, or to its original settled value if the promise was not handled (i.e. if the relevant handler `onFulfilled` or `onRejected` is not a function).

`Promise.prototype.finally()`

Appends a handler to the promise, and returns a new promise that is resolved when the original promise is resolved. The handler is called when the promise is settled, whether fulfilled or rejected.

FAQ

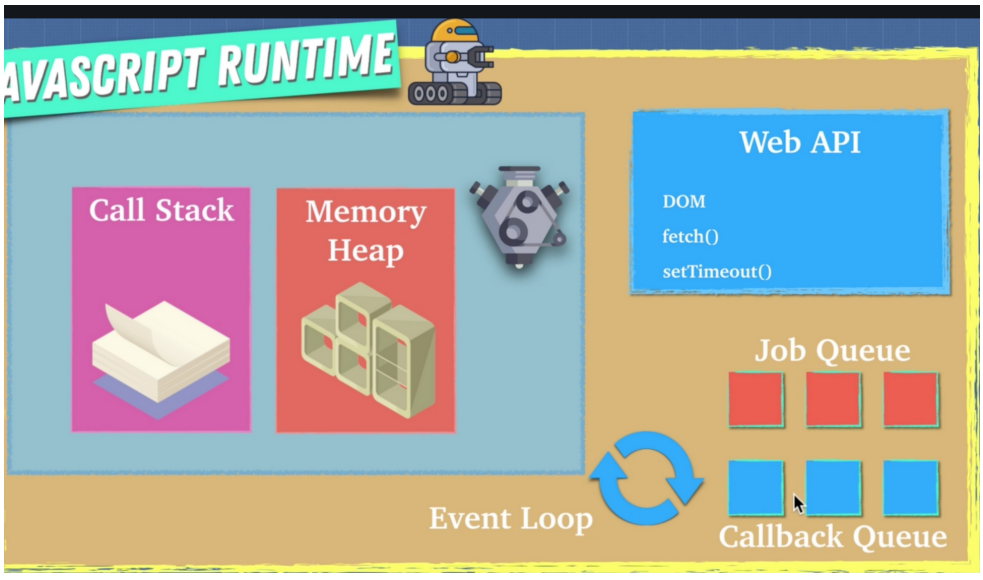
Callbacks, States, then().. all are confusing

It's simple. Let me break it down like this.

- While Executing any Promise, it has three states **(Pending, Resolved/Fulfilled, Rejected)**
- **And** Every Promise Accepts a function with two callbacks **(Resolve and Reject),** Which helps to return the data from the Promise.
- And we use **then()** to get the data from **resolve()** callback and **catch()** for **reject()**

Does JsEngine handle Promise ?

Similar to **Callback Queue,** there is an other queue for promises called as **Job Queue / Microtask Queue** which is managed by browser



FAQ

Why is a Promise executed before SetTimeout

Promises (put in **Job queues)** are given priority over setTimeout (put in **Callback Queues**) by **Event Loop**

Are promises blocking, by default ?

Promises are handled by the Browser hence thy run in parallel and non-blocking. But if you want blocking nature in your promise code, try **Async/Await**

Refer this example to understand **Async/Await**

How to handle Error case in Async Await?

try/catch can be used for Error cases. *Refer above example*

Examples

#1 Simple example of a Promise

One way of writing would be

```javascript
function getPromise() {
  return new Promise((resolve, reject) => {
    resolve('Promise is Successful/Fulfilled');
  })
}

getPromise().then((resp) => {console.log(resp)})
//Promise is Successful/Fulfilled
```

or

```javascript
function getPromise() {
  return new Promise((resolve, reject) => {
    reject('Promise is Failed');
  })
}

getPromise().catch((resp) => {console.log(resp)})
//Promise is Failed
```

This is a frequent interview question on Promises

Examples

#2 Example of Promise usage with SetInterval/SetTimeout API

One way of writing would be

```javascript
function aPromise() {
    return new Promise(() => {
        setTimeout(() => console.log("Hello"), 1000); //1 second
    })
}

aPromise();
```

```javascript
function aPromise() {
    return new Promise(() => {
        setInterval(() => console.log("Hello"), 1000);
//1 second
    })
}

aPromise();
```

EXAMPLES

#3 EXAMPLE WITH PROMISE.ALL

One way of writing would be

```javascript
let dataTypes = ["posts", "comments", "photos", "albums"];

// function to make API calls
function fetchData(type) {
  return fetch(`https://jsonplaceholder.typicode.com/${type}`
).then(response =>
    response.json()
  );
}

/*
 * Using `Promise.all`
 * 💡 Resolves if all the requests are successful
 * 💡 Rejects immediately if any request is failed
 */
async function loadAll(arr) {
  let promises = [];

  for (let i = 0; i < arr.length; i++) {
    promises.push(fetchData(arr[i]));
  }
  return await Promise.all(promises);
}

loadAll(dataTypes).then(res => console.log(res));

```

Other Examples here

This is a frequent interview question on Promises

EXAMPLES

#4 EXAMPLE WITH PROMISE.RACE

One way of writing would be

```javascript
let dataTypes = ["posts", "comments", "photos", "albums"];

// function to make API calls
function fetchData(type) {
  return fetch(`https://jsonplaceholder.typicode.com/${type}`
).then(response =>
    response.json()
  );
}

/*
 * Using `Promise.all`
 * Resolves if all the requests are successful
 * Rejects immediatly if any request is failed
 */
async function parallel(arr) {
  let promises = [];

  for (let i = 0; i < arr.length; i++) {
    promises.push(fetchData(arr[i]));
  }
  return await Promise.race(promises);
}

parallel(dataTypes).then(res => console.log(res));
```

This is a frequent interview question on Promises

CALLBACKS

Problem with Async programming is returning data. As **Js engine is Single threaded** and async code is handled by browser, there must be a way for the dependent code to get data from the Async functions.

Callbacks are functions passed as parameters and returns the data from the async functions. They were the **ancestors of the PROMISES**. There are few **disadvantages** with Callbacks

- Huge chunk of code
- If nested Async function can lead to callback Hells

Callbacks were old way of handling the Async programming. Prefer not to use them

References

Promise.all with Async/Await

Let's say I have an API call that returns all the users from a databas...

taniarascia.com

HTTP://LATENTFLIP.COM/LOUPE

SarathSantoshDamaraju/
JS-Tips

Return the result of an asynchronous function in JS

flaviocopes.com

How to return the result of an asynchronous function in JavaScript

Find out how to return the result of an asynchronous function, promise based or callback based, using JavaScript

Here are the most popular ways to make an HTTP request in JavaScript

JavaScript has great modules and methods to make HTTP requests that can be used to send or receive data from a server side resource. In this article, we a...